

Impacto dos Parâmetros da Hierarquia de Memória no Desempenho de Aplicações: Um Estudo de Simulação com gem5 e Multiplicação de Matrizes

Artur Fernandes Braga de Menezes
Arquitetura de Computadores
PUC Minas
Brasil

Artur Rizzi Martinho
Arquitetura de Computadores
PUC Minas
Brasil

Guilherme Meyer Guimarães
Arquitetura de Computadores
PUC Minas
Brasil

Pedro Hosken Fernandes Guimarães
Arquitetura de Computadores
PUC Minas
Brasil

Breno Moreira Cortez
Arquitetura de Computadores
PUC Minas
Brasil

Resumo—O projeto da hierarquia de cache é um dos fatores mais críticos para o desempenho de processadores em cargas de trabalho intensivas em computação. Este artigo apresenta um estudo sistemático de simulação utilizando o gem5 para avaliar o impacto de quatro parâmetros da hierarquia de memória na execução de um benchmark de multiplicação de matrizes: tamanho da cache L1 de dados, tamanho da cache L2, tipo de memória principal e associatividade da cache L1 de dados. Os experimentos foram conduzidos no modo Syscall Emulation (SE) em uma CPU Timing x86 de núcleo único a 3 GHz com hierarquia clássica de dois níveis. Os resultados mostram que a capacidade da L2 possui o maior impacto no desempenho – a transição de 32 KiB para 64 KiB reduz a taxa de miss da L2 de 93,9% para 1,96% e proporciona um speedup de 17% – enquanto aumentar o tamanho da L1 além do limite do conjunto de trabalho reduz drasticamente a taxa de miss, mas oferece apenas 1,5% de ganho geral. Associatividade da L1D acima de 4-way e variação do tipo de memória principal produzem melhorias desprezíveis nas condições estudadas. Esses resultados fornecem orientações práticas para o dimensionamento de caches em aplicações com alto grau de computação aritmética.

Index Terms—gem5, hierarquia de cache, simulação de memória, multiplicação de matrizes, cache miss, IPC, arquitetura de computadores

I. INTRODUÇÃO

A hierarquia de memória permanece como um dos desafios centrais do projeto de processadores modernos. A diferença entre as frequências de clock da CPU e as latências de acesso à memória principal – frequentemente chamada de *memory wall* – obriga os arquitetos a ajustar cuidadosamente múltiplos níveis de cache para mitigar a degradação de desempenho [1]. Para **cargas de trabalho intensivas em computação, como álgebra linear densa, os padrões de localidade espacial e temporal dos acessos a dados interagem diretamente com a capacidade, o tamanho de bloco, a associatividade e a política de substituição**

da cache, tornando a escolha dos parâmetros da hierarquia um fator decisivo para se atingir alto throughput de instruções [2].

Ferramentas de simulação ciclo a ciclo como o gem5 [3] permitem que pesquisadores e estudantes explorem esse espaço de projeto sem precisar de mudanças no hardware físico. O gem5 oferece um framework modular e flexível que modela uma ampla gama de hierarquias de memória, modelos de CPU e ISAs, sendo amplamente utilizado em pesquisa arquitetural para analisar o comportamento de caches e subsistemas de memória [4].

Este trabalho apresenta um estudo de simulação controlada com o gem5 para responder às seguintes perguntas:

- Como o aumento do tamanho da cache L1 de dados afeta o desempenho e o comportamento de misses?
- Qual o impacto da variação do tamanho da cache L2 no desempenho?
- A atualização da tecnologia de memória principal (DDR3 vs. DDR4) produz ganhos mensuráveis?
- Como a associatividade da cache L1D influencia as taxas de miss e o tempo de execução?

Todos os experimentos utilizam o binário x86-matrix-multiply, disponível na biblioteca de recursos padrão do gem5, executado no modo SE em uma CPU Timing x86 de

núcleo único. Esse workload exhibe padrões de acesso à memória regulares baseados em stride, característicos de operações matriciais densas, estressando a hierarquia de cache de dados de forma previsível e analisável. As principais contribuições deste artigo são:

- 1) Uma infraestrutura experimental reproduzível e de código aberto, construída sobre o gem5 v25.1.0.0, para avaliação de parâmetros de hierarquia de cache;
- 2) Caracterização quantitativa da sensibilidade de desempenho ao tamanho da L1, ao tamanho da L2, à tecnologia de memória e à associatividade de cache sob um workload de multiplicação de matrizes;
- 3) Identificação dos limiares de conjunto de trabalho a partir dos quais os parâmetros de cache deixam de proporcionar ganhos de desempenho significativos.

O restante do artigo está organizado da seguinte forma. A Seção II discute os trabalhos correlatos. A Seção III descreve o ambiente de simulação e o projeto experimental. A Seção IV apresenta e analisa os resultados. A Seção V conclui e aponta direções futuras.

II. TRABALHOS CORRELATOS

O simulador gem5 foi introduzido por Binkert et al. [3] como uma fusão dos simuladores M5 e GEMS, fornecendo uma plataforma de simulação altamente configurável em modo full-system e SE. A ferramenta tem sido continuamente atualizada; Lowe-Power et al. [4] descreveram a versão gem5 20.0, que introduziu a *biblioteca padrão do gem5* utilizada neste trabalho para compor hierarquias de memória por meio de componentes Python de alto nível.

O dimensionamento de caches e sua relação com o desempenho de workloads tem sido extensivamente estudado na literatura de arquitetura. Cantin e Hill [5] realizaram uma exploração do espaço de projeto de configurações de cache utilizando benchmarks SPEC CPU e demonstraram que o benefício relativo de aumentar a capacidade da cache é fortemente dependente do workload. Seus resultados estão alinhados com a observação deste estudo de que os ganhos de desempenho se estabilizam assim que o conjunto de trabalho cabe inteiramente na cache.

Em relação à multiplicação de matrizes especificamente, Lam et al. [6] demonstraram que estratégias de tiling orientadas a cache podem reduzir drasticamente os misses em álgebra linear densa, e que mesmo com uma implementação de laço triplo ingênua, o comportamento de miss na L1 é governado pela dimensão da matriz em relação à capacidade da cache. O benchmark x86-matrix-multiply utilizado neste trabalho segue uma estrutura padrão de laço triplo, tornando-o um proxy adequado para avaliação de estresse de cache.

O efeito da associatividade de cache foi analisado por Sugumar e Abraham [7], que mostraram que, para a maioria dos workloads, aumentar a associatividade além de 4-way ou 8-way produz retornos decrescentes, pois os misses por conflito representam uma fração pequena do total de misses. Nossos resultados confirmam essa observação para o workload de multiplicação de matrizes estudado.

Mais recentemente, estudos baseados em simulação com gem5 para exploração de hierarquia de memória foram reportados em contextos educacionais e de pesquisa [8], validando o gem5 como plataforma confiável para experimentos arquiteturais e motivando seu uso em projetos acadêmicos como este.

III. METODOLOGIA / ARQUITETURAS SIMULADAS

A. Ambiente de Simulação

Todas as simulações foram executadas em macOS utilizando o gem5 versão v25.1.0.0 (commit 7a2b0e41), compilado com Clang 16.0.6 e Python 3.14.3 via SCons 4.10.1. A configuração base para todos os experimentos é resumida na Tabela I.

Tabela I
CONFIGURAÇÃO BASE DA SIMULAÇÃO

Parâmetro	Valor
Simulador	gem5 v25.1.0.0 (modo SE)
ISA	x86
Modelo de CPU	Timing (em ordem)
Frequência de clock	3 GHz
Número de núcleos	1
Tamanho de linha de cache	64 bytes
Memória principal	DDR3-1600, 1 GiB
Hierarquia	L1 privada, L2 privada (clássica)
Workload	x86-matrix-multiply

O componente PrivateL1PrivateL2CacheHierarchy da biblioteca padrão do gem5 foi utilizado em todos os experimentos de cache, fornecendo caches L1 de instrução (L1I) e L1 de dados (L1D) separadas, suportadas por uma cache L2 unificada. Os scripts de configuração em Python (configs/cache_exp.py, configs/assoc_exp.py, configs/memory_exp.py) parametrizam tamanhos de cache e tipos de memória via argumentos de linha de comando, permitindo execução em lote automatizada.

B. Workload

O benchmark x86-matrix-multiply é um kernel padrão de multiplicação de matrizes distribuído como parte da biblioteca de recursos do gem5. Ele realiza a multiplicação de matrizes densas $N \times N$ utilizando uma estrutura de laço triplo (i, j, k) , conhecida por exibir alta localidade espacial no laço interno para as duas matrizes-fonte e menor localidade temporal para a matriz resultado. Todas as 26.728.189

instruções foram simuladas até a conclusão em cada execução.

C. Projeto Experimental

Quatro experimentos independentes foram conduzidos, cada um variando um parâmetro por vez:

Experimento A – Tamanho da Cache L1: Os tamanhos de L1D e L1I foram variados juntos entre {16 KiB, 32 KiB, 64 KiB}, com L2 fixada em 256 KiB. A baseline é a configuração de 16 KiB.

Experimento B – Tamanho da Cache L2: O tamanho da L2 foi variado entre {32 KiB, 64 KiB, 128 KiB, 256 KiB}, com L1D e L1I fixadas em 16 KiB. A baseline é a configuração de 32 KiB.

Experimento C – Tipo de Memória Principal: Três tecnologias de memória DDR foram comparadas – DDR3-1600, DDR3-2133 e DDR4-2400 – utilizando uma hierarquia fixa de L1D/L1I = 16 KiB e L2 = 32 KiB (intencionalmente pequena para estressar o caminho à memória principal). A baseline é DDR3-1600.

Experimento D – Associatividade da L1D: A associatividade da L1D foi variada entre {2-way, 4-way, 8-way} com tamanho de L1D/L1I fixado em 16 KiB e L2 fixada em 64 KiB, utilizando o componente

PrivateL1SharedL2CacheHierarchy (l1i_assoc = 8, l2_assoc = 16). A baseline é 2-way.

D. Métricas Analisadas

As seguintes métricas foram extraídas dos arquivos stats.txt gerados pelo gem5:

- IPC (Instruções Por Ciclo) e CPI (Ciclos Por Instrução);
- Speedup em relação à baseline de cada experimento;
- Contagens de hits e misses da L1D, L1I e L2;
- Taxa de miss = misses / total de acessos;
- MPKI (Misses Por Kilo-Instruções) = (misses / simInsts) × 1000;
- Latência média de miss (em ticks de simulação, onde 1 tick = 1 ps a 1 THz de simFreq).

IV. AVALIAÇÃO DOS RESULTADOS

A. Experimento A: Tamanho da Cache L1

A Tabela II apresenta os resultados para a variação do tamanho da cache L1. A transição de 16 KiB para 32 KiB produz uma melhoria desprezível (speedup = 1,0001, Δ IPC < 0,001), com a taxa de miss da L1D diminuindo marginalmente de 0,412% para 0,411%. Isso indica que o conjunto de trabalho de dados do benchmark ultrapassa levemente 16 KiB, mas não 32 KiB em termos de comportamento de conflitos.

O salto para 64 KiB, porém, provoca uma mudança qualitativa: a taxa de miss da L1D colapsa de 0,411% para apenas 0,004%, reduzindo o MPKI da L1D de 2,37 para 0,021. Isso indica que 64 KiB é suficiente para conter todo o conjunto de trabalho de

dados ativo, eliminando virtualmente todos os misses por capacidade e conflito no nível L1. O speedup correspondente é de 1,016 (1,6% mais rápido) e o CPI melhora de 2,000 para 1,969.

Tabela II

EXPERIMENTO A – VARIAÇÃO DO TAMANHO DA L1 (L2 = 256 KiB)

TM = taxa de miss.

A anomalia na taxa de miss da L2 com 64 KiB (subindo para 23,6%) merece atenção: com a L1D absorvendo quase todas as requisições de dados, os poucos misses que escapam para a L2 são majoritariamente misses frios que não podem ser servidos ali, resultando em uma alta taxa de miss da L2 sobre um número absoluto de misses muito pequeno (995 misses). Trata-se de um artefato estatístico do volume reduzido de acessos à L2, não de uma degradação do comportamento da L2.

B. Experimento B: Tamanho da Cache L2

A Tabela III evidencia o forte impacto do tamanho da L2. Com L2 = 32 KiB, a taxa de miss da L2 atinge 93,9%, significando que quase todos os misses da L1D escalam até a memória principal. Isso produz o pior IPC (0,427) e o maior CPI (2,342) entre todos os experimentos.

Tabela III

EXPERIMENTO B – VARIAÇÃO DO TAMANHO DA L2 (L1D = L1I = 16 KiB)

L2	IPC	CPI	Speedup	Hits L2	TM L2	MPKI L2
32 KiB	0,427	2,342	1,000	4.192	93,9%	2,410
64 KiB	0,500	2,002	1,170	67.249	1,96%	0,050
128 KiB	0,500	2,000	1,171	67.593	1,46%	0,038
256 KiB	0,500	2,000	1,171	67.603	1,45%	0,037

O aumento da L2 para 64 KiB proporciona um speedup de 17% e reduz a taxa de miss da L2 para 1,96%, pois 64 KiB é suficiente para armazenar o conjunto de trabalho matricial que repetidamente sofre miss na L1D de 16 KiB. Além disso, novos aumentos (128 KiB e 256 KiB) trazem ganhos adicionais marginais (Δ speedup < 0,001), evidenciando um claro ponto de saturação do conjunto de trabalho em aproximadamente 64 KiB para este benchmark.

Esses resultados destacam que o dimensionamento da L2 é a alavanca dominante para este workload: uma única duplicação de capacidade (32→64 KiB) libera a maior parte da melhoria de desempenho alcançável, enquanto duplicações subsequentes trazem benefício desprezível.

C. Experimento C: Tecnologia de Memória Principal

A Tabela IV compara três configurações de memória DDR. Neste experimento, a L2 é intencionalmente pequena (32 KiB) para produzir uma alta taxa de acessos à DRAM (≈ 64.400 misses na L2 por execução). Apesar disso, as diferenças de desempenho entre os tipos de memória são mínimas: DDR32133 proporciona speedup de 0,79% sobre DDR3-1600,

L1	IPC	CPI	Speedup	TM L1D	MPKI L1D	TM L2
16 KiB	0,500	2,000	1,000	0,412%	2,38	1,45%
32 KiB	0,500	2,000	1,000	0,411%	2,37	1,48%
64 KiB	0,508	1,969	1,016	0,004%	0,02	23,6%

e

DDR4-2400 apenas 0,28%.

Tabela IV
XPERIMENTO C – TIPO DE MEMÓRIA PRINCIPAL (L1D = L1 = 16 KiB, L2 = 32 KiB)

Memória	IPC	CPI	Speedup	Lat. Média L2 (ticks)
DDR3-1600	0,427	2,342	1,000	52.512
DDR3-2133	0,430	2,324	1,008	49.926
DDR4-2400	0,428	2,336	1,003	51.627

A baixa sensibilidade ao tipo de memória pode ser explicada por dois fatores. Primeiro, mesmo com uma L2 de 32 KiB, o número total de acessos à DRAM (64.400) representa uma fração pequena das 26,7 milhões de instruções totais. Segundo, a CPU Timing trava todo o pipeline a cada miss de cache, portanto a redução da latência de DRAM melhora o tempo de execução apenas proporcionalmente à fração de ciclos aguardando a DRAM. Com 53–62 milhões de ciclos totais e 64.400 requisições DRAM cada com ≈ 50.000 ticks (≈ 50 ns), a fração DRAM-bound é relevante, mas não dominante o suficiente para que diferenças de largura de banda sejam perceptíveis.

Esse resultado ressalta uma percepção arquitetural importante: atualizar a tecnologia de memória isoladamente é ineficaz quando o gargalo primário é a capacidade de cache – a intervenção correta é primeiro dimensionar adequadamente a hierarquia de cache.

D. Experimento D: Associatividade da Cache L1D

A Tabela V mostra o impacto da associatividade da L1D. A configuração 2-way produz taxa de miss (2,649%) e MPKI (15,27) dramaticamente maiores em comparação com 4-way (0,426% e MPKI 2,46) e 8-way (0,412% e MPKI 2,38). A transição de 2-way para 4-way proporciona speedup de 8,3%, enquanto a de 4-way para 8-way agrega apenas 0,05%.

Tabela V
EXPERIMENTO D – ASSOCIATIVIDADE DA L1D (L1D = 16 KiB, L2 = 64 KiB)

Assoc.	IPC	CPI	Speedup	TM L1D	MPKI L1D
--------	-----	-----	---------	--------	----------

2-way	0,461	2,169	1,000	2,649%	15,27
4-way	0,499	2,002	1,083	0,426%	2,46
8-way	0,500	2,001	1,084	0,412%	2,38

A alta taxa de miss com associatividade 2-way sinaliza um número expressivo de misses por conflito: o padrão de acesso da multiplicação de matrizes mapeia múltiplas linhas de matrizes para os mesmos conjuntos de cache quando se utiliza baixa associatividade e dimensão matricial potência de dois, um caso patológico bem conhecido para caches mapeadas diretamente e 2-way [6]. Aumentar a associatividade para 4way resolve a maioria dos conflitos, e 8-way proporciona melhoria marginal adicional porque os misses por capacidade, e não por conflito, dominam os eventos de miss restantes.

O fato de 4-way oferecer praticamente o mesmo desempenho que 8-way (diferença de speedup de 0,001) fornece forte evidência de que o ponto ótimo de associatividade para este workload em 16 KiB é 4-way, representando uma troca favorável entre área, consumo de energia e desempenho.

E. Síntese dos Resultados

A Tabela VI resume os ganhos de speedup máximos obtidos em cada experimento. A conclusão dominante é que a *capacidade da L2* é o parâmetro de maior impacto para este workload, seguida da *associatividade da L1D* (quando muito baixa), da *capacidade da L1D* (apenas acima de um limiar de conjunto de trabalho) e, por fim, da *tecnologia de memória principal* (desprezível quando o dimensionamento de cache é inadequado).

Tabela VI
SPEEDUP MÁXIMO POR EXPERIMENTO (RELATIVO À BASELINE DE CADA UM)

Experimento	Speedup Máximo
B: L2 32→64 KiB	1,170
D: Assoc. L1D 2→4-way	1,083
A: L1D 16→64 KiB	1,016
C: Memória DDR3-1600→2133	1,008

V. CONCLUSÕES

Este artigo apresentou um estudo baseado em simulação dos parâmetros da hierarquia de memória para um workload de multiplicação de matrizes, utilizando o gem5 v25.1.0.0 no modo SE em uma CPU Timing x86. Quatro dimensões foram avaliadas: tamanho da cache L1, tamanho da cache L2, tecnologia de memória principal e associatividade da cache L1D.

Os principais achados são:

- 1) O tamanho da L2 é a alavanca de desempenho dominante. A transição de 32 KiB para 64 KiB reduziu a taxa de miss da L2 de 93,9% para 1,96% e proporcionou speedup de 17%. Além desse limiar, os ganhos se estabilizam porque o conjunto de trabalho é completamente capturado pela L2.

- 2) O tamanho da L1D importa apenas próximo aos limiares do conjunto de trabalho. Dobrar a L1D de 16 para 32 KiB não produziu efeito mensurável; dobrar para 64 KiB reduziu o MPKI da L1D de 2,38 para 0,02, proporcionando speedup de 1,6%. Isso ressalta a importância de dimensionar a capacidade da cache ao conjunto de trabalho real, e não simplesmente dobrar a capacidade.
- 3) A tecnologia de memória contribui marginalmente quando a capacidade de cache é insuficiente. Com uma L2 de 32 KiB forçando tráfego à DRAM, DDR32133 ofereceu apenas 0,79% de melhoria sobre DDR31600, e DDR4-2400 apenas 0,28%. O passo correto é primeiro corrigir a hierarquia de cache.
- 4) A associatividade da L1D é crítica em valores baixos. Associatividade 2-way causou intensa inundação de misses por conflito (MPKI = 15,27) para este padrão de acesso. A migração para 4-way resolveu a maioria dos conflitos e proporcionou speedup de 8,3%. 8-way não trouxe benefício adicional, sugerindo que 4-way é o ponto Pareto-ótimo para este workload.

Trabalhos futuros incluem: (i) extensão do estudo a configurações multi-core para avaliar o impacto de L2 compartilhada vs. privada no tráfego de coerência; (ii) variação do tamanho de linha de cache para analisar sua interação com padrões de acesso baseados em stride; (iii) utilização de modelos de CPU fora-de-ordem (O3) para avaliar se o paralelismo em nível de memória altera a sensibilidade aos parâmetros de cache; e (iv) uso de workloads otimizados para cache (e.g., multiplicação de matrizes com tiling) para comparar o comportamento de misses com a variante ingênua de laço triplo.

AGRADECIMENTOS

Os autores agradecem à comunidade gem5 por manter uma infraestrutura de simulação de código aberto e aos professores da disciplina pela orientação ao longo do projeto.

REFERÊNCIAS

- [1] W. A. Wulf e S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, n. 1, pp. 20–24, mar. 1995.
- [2] J. L. Hennessy e D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6.^a ed. Morgan Kaufmann, 2019.
- [3] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill e D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, n. 2, pp. 1–7, ago. 2011.
- [4] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, S. Bharadwaj, G. Black et al., "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.
- [5] J. F. Cantin e M. D. Hill, "Cache performance for selected SPEC CPU2000 benchmarks," *ACM SIGARCH Computer Architecture News*, vol. 29, n. 4, pp. 13–18, set. 2001.

- [6] M. S. Lam, E. E. Rothberg e M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Santa Clara, CA, abr. 1991, pp. 63–74.
- [7] R. A. Sugumar e S. G. Abraham, "Efficient simulation of caches under optimal replacement with applications to miss characterization," in *Proc. 1993 ACM SIGMETRICS Conf. Measurement and Modeling of Computer Systems*, Santa Clara, CA, mai. 1993, pp. 24–35.
- [8] J. Akram e L. Sawalha, "A survey of computer architecture simulation techniques and tools," *IEEE Access*, vol. 7, pp. 9800–9828, 2019, doi: 10.1109/ACCESS.2019.2930584.

Link para o repositório onde foram conduzidos os experimentos:
<https://github.com/guimeyer2/gem5-memory-hierarchy-study>